

Inside the Simple Builder

One paragraph of brief becomes three complete, WordPress-ready page concepts — copy written, imagery placed, blocks composed from the registered DST library. This is how.

4 STEPS IN THE SIMPLE BUILDER	1 BUTTON DOES THE WHOLE FIRST PASS	156 REGISTERED DST PATTERNS	3 ISOLATED CONCEPT WORKSPACES
---	--	---	---

A strategist writes one paragraph. A client sees three finished pages.

Everything between those two facts is this application. It reads the brief, decides which of 156 registered DST patterns the page should be built from, writes the copy for every band, finds and places the imagery, and renders the result as a live page you can edit — then exports it as block JSON a WordPress plugin turns into real, editable Gutenberg blocks.

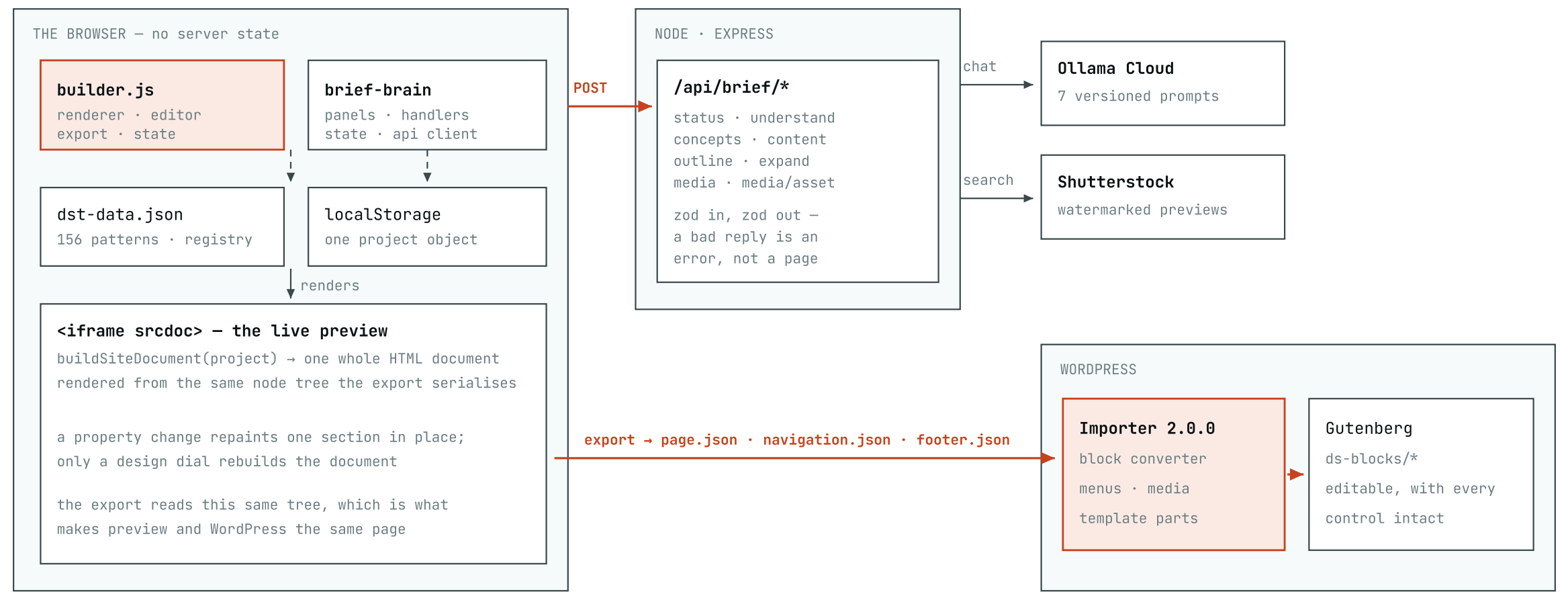
It runs **entirely in the browser**. The only server is a thin proxy in front of a language model and a stock-image API: no database, no build step per project, no session state. A project is one JSON object in `localStorage`.

The design constraint that shapes everything

The preview is not a mock-up of the output. It is rendered from *the same node tree* the export serialises. When they disagree, that is a bug with a name — and there is a test for it.

Two front doors, one engine

The **simple builder** is four steps and plain language. The **advanced builder** is five steps over the same project, exposing every registered DST attribute. Switching between them loses nothing, because there is only one project.



One request path (accent) carries the brief out and the concepts back; a second carries the finished project into WordPress. Everything else stays in the tab. Dashed lines are reads.

Four steps, and the first one does most of the work.

STEP 01**Brief & Direction**

One paragraph in. Three concepts, all the copy and all the imagery out.

STEP 02**Page flow**

Which bands the page is made of, and in what order.

STEP 03**Modules**

Per-band content, media and layout — in plain language.

STEP 04**Review & export**

Preflight, then the artifacts WordPress consumes.

The brief arrives as a paragraph, a PDF, or both.

Typed

A textarea, capped at 16,000 characters. The counter and the button's gate both measure the *whole* brief.

Attached

Drop a PDF, `.docx`, `.rtf` or any text file anywhere on the window. It is read **in the browser** — nothing is uploaded — and shows as a named attachment, not as pasted text.

`briefSourceText()`

One function decides what "the brief" is: the paragraph, then each document announced by its own file name. Every consumer reads it, so they cannot disagree.

The PDF reader is written from scratch and has no dependency:

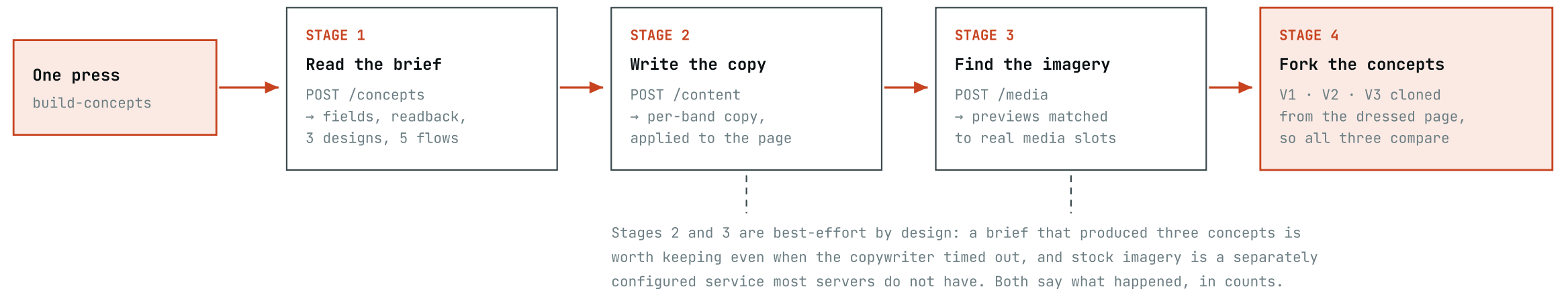
`DecompressionStream` for the Flate streams, a table walk for `/ObjStm` object streams, and a `/ToUnicode` CMap resolver for `Identity-H` CID fonts — which is what Word and Chrome actually export. A PDF stores no spaces and no lines, so word gaps are inferred from kerning and line breaks from pen position.

→ What the reader handles

WinAnsi TrueType and Identity-H CIDFontType2 in the same file; PDF 1.5+ compressed object streams; DOCX as a ZIP with a deflated `word/document.xml`; RTF control-word groups; HTML.

→ What it refuses, by name

A scan. If less than 55% of the extracted characters are letters, it says so and asks for the text instead of handing the model noise.



Order is the whole point: the page is dressed with copy and pictures *before* it forks, so the three concepts differ only in design and are genuinely comparable.

What the panel reports

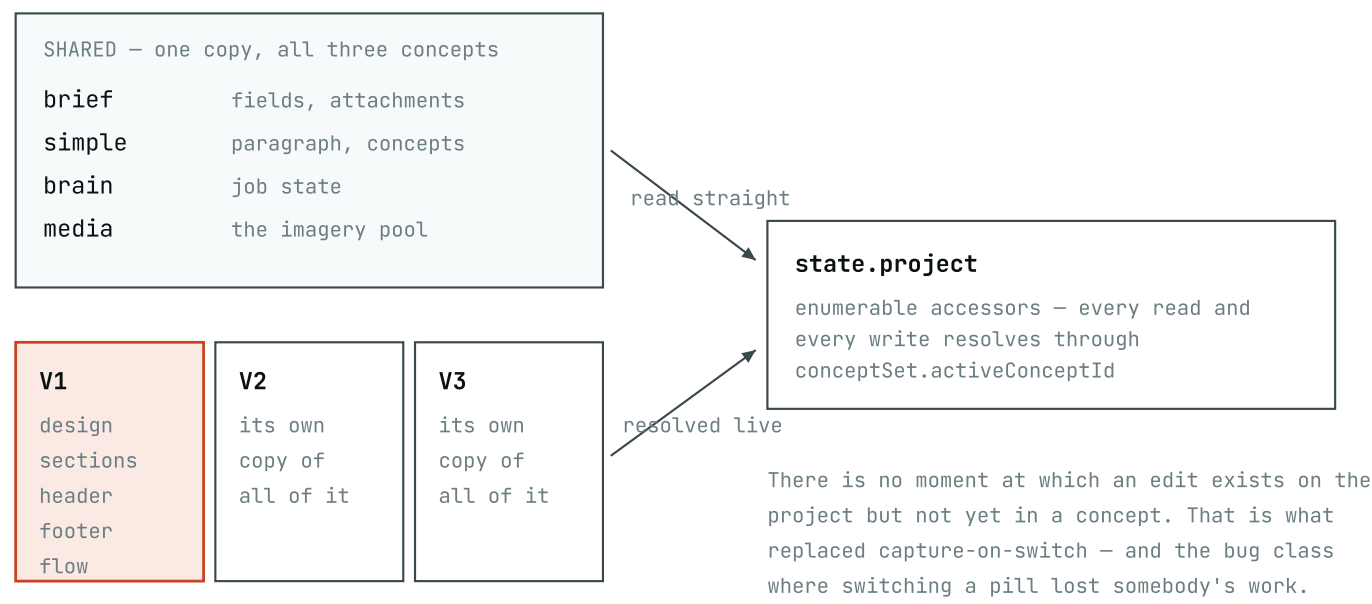
Concepts built, sections written, pictures placed — and a note for anything that did not happen, rather than a toast that has already gone.

Staleness is tracked

Editing the paragraph or removing an attachment marks the concepts stale, with the reason on the panel.

No second button

Applying copy and applying imagery used to be two more presses. They are stages now; the buttons are gone from the code, not hidden.



A concept is a *design*, not a page. Switching V1/V2/V3 replaces the design slice and can never disturb content, flow or globals.

Three workspaces, one brief.

Each concept holds its own design, sections, header, footer and flow. The brief, the imagery pool and the job state are shared, because they belong to the project rather than to a proposal.

What the pills guarantee

Switching a concept is not an undoable edit, and undo is per concept. A design edit stays with the concept it was made on. All of it survives a reload.

Where the design comes from

Nine numeric dials – density, imagery, expressiveness, motion, measure and the rest – plus a palette, two typefaces and a button family. A concept is those values; the page is what they resolve to.

Which bands the page is made of, and why those.

A flow is an ordered list of section families — `hero · text · split · cards · stats · testimonial · faq · cta` and so on. The library holds 35 of them, and the model ranks every one against the brief, returning the five that fit with a reason for each.

The strategist can also describe the page in their own words; that goes to `/outline` and comes back as a custom sequence.

The flow decides families. Something else decides patterns.

Choosing a flow does not choose a design. Each family has many registered patterns, and which one a band gets is scored separately — see the next slide.

FAMILY	PATTERNS	WHAT IT IS
cards	34	Repeating offer or capability cards
contact	16	Form bands, ten of them embedding Gravity Forms
faq	12	Accordion question sets
hero	11	Page openers, mostly photo-backed
timeline	11	Ordered process or history
split	9	Media beside text
stats	9	Metric bands with illustrative figures
team · slider	7 · 7	People, and horizontal sets
testimonial · cta · text	6 each	Quotes, the closing ask, statements
blog · pricing · tabs · logo	5 · 4 · 4 · 4	Resources, tiers, panels, proof strip

156 REGISTERED PATTERNS	52 DST BLOCK TYPES IN THE REGISTRY	1,270 BLOCK ATTRIBUTES THE EXPORT KNOWS	58 DECORATIVE MOTIFS
---	--	---	--

Every pattern is a real WordPress export — a tree of `ds-blocks/*` nodes with their attributes. Picking one is arithmetic and it is inspectable: the catalogue describes each pattern's look, what it is best for and what it holds; the brief and the nine design dials describe what this page needs. Matching the two produces a ranking with reasons.

Two properties matter as much as the ranking. It is **deterministic** — the same brief and dials give the same page every time, because a layout that reshuffles on each press is a slot machine, not a recommendation. And it **varies with the concept**, because the dials do: the calm concept gets contained bands, the bold one reaches for the photo-backed and full-bleed members of the same families.

Self-description is derived, not shipped

Each pattern's item counts, capability flags and one-line description are recomputed at boot *from its own tree*. They used to be stored, and they had drifted on two thirds of the library — which mattered, because pattern selection reads them.

The registry comes from the theme

`npm run sync:registry` reads the digitalsilk theme's own `block.json` manifests plus the attributes its editor HOCs inject. The export validates against that, so an imported block is editable with every control the builder used.

Per-band editing, in plain language.

Content

Pretitle, title, subtitle, body, the repeated items and the buttons. Typing patches the band in the live document as you type.

Media

Every slot the pattern actually renders, the found imagery, project assets and the placeholder library. Drag a tile onto the band to place it.

Layout + effects

Background, content width, spacing above and below, items per row, arrival effect, overlay and a decorative motif.

Every control lands in the export

A property you can change in the builder and cannot find in WordPress is worse than a missing control: the page was approved with it. So a test drives *every* binding the module editor renders — all 19 families, both views, a real input event each — and asserts the exported artifact changed.

It found five controls that changed nothing, each fixed at its cause. One was withdrawn rather than fixed: `c-heading` has a single alignment pair, so a separate supporting-text alignment had no attribute to become. It moved the preview and vanished on export.

Editing never moves the preview

A rebuilt `srcdoc` is a new document: it opens at scroll 0 and the restore walks it back down, taking the band you are editing with it. So a property change swaps *one section* in place and cancels the rebuild queued behind it. Repaints coalesce to one per frame, so dragging a slider patches the band per frame rather than rebuilding the page per pixel.

Two counters make it provable

`paint.painting()` and `paint.rebuilt()`. A repaint that silently degraded into a rebuild is the failure this exists to prevent, and a test asserting only "the change landed" cannot tell the two apart.

BEFORE – every change rebuilt the document



NOW – one section is swapped in place



A design dial still rebuilds, on purpose: it rewrites the document stylesheet and is not one band. A torn-down frame or a hidden band also falls back – moving the page is better than showing a stale one.

MEASURED, NOT ASSUMED

frame load count stays at 0 · scroll position unchanged to the pixel · paint.rebuild() stays at 0
A repaint that silently degraded into a rebuild is the failure this exists to prevent.

`reveal: true` is what makes the swap invisible: it marks every viewport effect in the new section as already seen, so the band does not replay its entrance under the cursor.

Preflight first, then four artifacts.

ARTIFACT	WHAT IT CARRIES
page.json	The page: every band as a DST node tree, plus the resolved theme
navigation.json	The header as a real <code>dst-navigation</code> tree, plus the menus to build
footer.json	The footer as a real <code>dst-footer</code> tree, three sections
website.html	The whole rendered page, standalone — for review and sign-off
complete-project.json	All of the above in one file, plus all three concepts

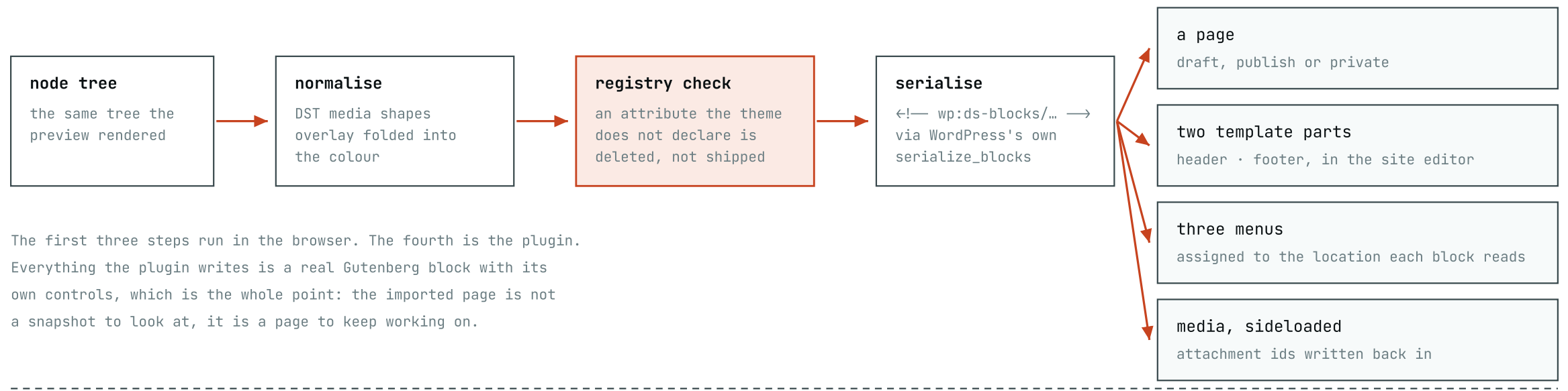
The preflight gates

One clear hero · a complete sequence · pattern provenance intact · every image has alt text · the palette passes contrast · *and* the rendered page passes contrast, which is a different question with a different answer.

The theme travels with the page

Not the slider numbers — the **resolved token set**: radii, container widths, vertical rhythm, the full type scale, motion duration and distance. WordPress consumes the exact values the preview used rather than reconstructing an approximation.

Every concept can be exported on its own, or all three at once. The concept JSON round-trips: it imports back into the advanced builder with every edit, field and concept intact.



The first three steps run in the browser. The fourth is the plugin. Everything the plugin writes is a real Gutenberg block with its own controls, which is the whole point: the imported page is not a snapshot to look at, it is a page to keep working on.

WHAT MAKES IT EXACT

The preview and the export read the same node tree, and every attribute in it is looked up in the theme’s own block manifests before it ships. 194 blocks and 1,080 attributes checked; nothing the theme cannot render leaves the builder.

Overlay strength is folded into the colour, because the theme carries opacity inside the colour and declares no opacity attribute – exported beside it, every scrim landed at full strength.

The registry check is the load-bearing step. An attribute WordPress does not know is kept in the markup and silently ignored – a setting somebody made that the page does not have.

Seven prompts, each a versioned file in `server/ai/prompts/` rather than a string in the code. Every response is validated by a Zod schema before it reaches the page: a reply that does not match is an *error with a code*, not a half-built page.

PROMPT	ITS ONE JOB
brief-architect	Read the paragraph into brief fields and a readback
concept-designer	Three design directions, each with dials and a palette
flow-outline	Rank the 35 flows; or turn a described page into a sequence
content-writer	The copy for every band, in order, respecting item counts
media-director	What each media slot should show
media-search	The search terms to find it
brief-expander	Split a pasted brief into the advanced builder's fields

Never invent verifiable facts

No prices, client names, years in business, certifications, awards, review counts or staff names. Where a section structurally needs one, the model writes an instruction the strategist replaces.

One narrow exception, stated out loud

A stats band reading "add the measured figure" three times is a template, not a concept. So the model writes an *illustrative* figure in the unit the trade uses — `2,000 km`, `48 hrs`, `12 sites` — and the band's own body says the figures are illustrative and to be confirmed.

Degradation is visible

No provider configured, a timeout, a refusal: each returns a typed error and the panel says which. A built-in planner covers the flow ranking so the page still builds, and the panel labels it as the built-in planner rather than the model.

LAYER	CHOICE	WHY
UI	vanilla JS	No framework. The renderer is a pure function from project to HTML string, which is what lets the preview and the export share it.
Build	Vite 7	ES modules in dev, one bundle out. No transpiler config.
Server	Express 5	Eight routes. Holds credentials and schemas; owns no state.
Validation	Zod 3	Every model reply, in and out. Shared with the client.
Model	Ollama Cloud	Cloud only, by policy. JSON-schema-constrained decoding.
Imagery	Shutterstock	Watermarked previews only; nothing is licensed automatically.
Storage	localStorage	One project object, debounced, flushed on page-hide.
Documents	none	PDF, DOCX, RTF and ZIP read with platform APIs only.
Tests	Vitest · Playwright · PHP	Unit, real-browser, and the importer against the theme.
Import	PHP plugin	Source in the repo, artifact built by a script.

Four runtime dependencies.

`express`, `zod`, `dotenv`, `playwright`. That is the whole list. The PDF reader, the ZIP reader, the deflate, the diagrams and the export are all platform APIs.

How the monolith is organised

`builder.js` is one IIFE with **21 numbered layers**, each a dated commentary on what it changed and why. A layer reassigns the bindings it needs to change — `renderPreview`, `updateBinding`, `syncSectionNode` — so the history of every behaviour is readable in one file, in order.

What lives in `shared/`

19 ES modules used by the browser, the server *and* the tests: the design dials, the palette repair, the concept workspace, the brief schemas, the document readers, the style compiler. One statement of each rule.

328 REAL-BROWSER TESTS	301 UNIT TESTS	36 WORDPRESS IMPORTER ASSERTIONS	0 ATTRIBUTES THE THEME DOES NOT KNOW
--	--	--	--

Several of these are instruments rather than coverage. Each asserts a *property of the product* that a reasonable change could break silently, and each states in its own comment what went wrong before it existed.

01 Every editor property reaches the export
All 19 families, both views, a real input event per control, every select option, both slider directions.

02 Every pattern describes itself truthfully
Counts, flags and description checked against the pattern's own tree — per pattern, not in aggregate.

03 Nothing points at the exporting site
No rendered URL, link or attribute names the staging install the patterns came from.

04 The rendered page is readable
Not the palette — the pixels, measured against each line's own ground, with a negative control that proves the instrument still fails.

05 The preview never moves
Frame load count stays at zero and the scroll position is unchanged *to the pixel*.

06 The export exists in the theme
Every block and attribute looked up in the theme's own manifests. Skipped, not faked, when no theme is checked out.

The fixtures are generated, not remembered
`npm run record:wordpress` drives the real app in a real browser and writes what the Export button produces. They were recordings, and stale — so the importer tests passed against a page the builder no longer made.
A fixture that does not match the product is worse than none: it reports success

PATH	WHAT IS IN IT
src/runtime/builder.js	The application: state, renderer, editor, preview, export. 21 layers, in order.
src/features/brief-brain/	The AI panels, the job state, the request builders and the API client.
src/data/dst-data.json	156 patterns, the component registry, flows, archetypes, motifs.
shared/	19 modules used by browser, server and tests alike.
server/	Express app, eight routes, the provider, seven prompt files.
patternsSBS/	The 169 source pattern exports the library is ingested from.
wordpress-plugin/	The importer's PHP source.
tests/	36 browser specs, 14 unit specs, the PHP importer suite, fixtures.
scripts/	Registry sync, pattern cleaning and ingest, fixture recording, theme verification, plugin build.
RELEASE-NOTES.md	Every change, with the reasoning. Start here.

Three things to know before extending it

The IIFE boundary is real

Code before `builder.js:729` cannot see functions declared inside the layered section. Calling one from a listener throws a `ReferenceError` the browser swallows — the mutation applies, the repaint does not, and nothing is reported. Add to a layer, not above one.

The renderer is the contract

The preview and the export read the same node tree. Any change to one that is not a change to the other is the bug the whole verification stack exists to catch.

The theme is the authority

Not the pattern files, not the registry snapshot: the theme's `block.json` manifests plus the attributes its editor HOCs inject. Re-sync when the theme moves, and the export follows.

One paragraph in. A page you can keep working on out.

TODAY

Four steps, one press, three comparable concepts. Real DST blocks, editable in WordPress with every control intact.

Run it `npm install • npm run dev • http://127.0.0.1:5173`
Check it `npm run test:all • npm run test:handoff • npm run verify:export`
Ship it `npm run build • npm run build:plugin`

VERIFIED

665 automated checks, every exported block and attribute looked up in the theme that has to render it.

NEXT

Review & share with public URLs and frozen snapshots; the megamenu family; server-side project storage.